

অপারেটিং সিস্টেমের ভূমিকা

(Introduction to Operating Systems)

প্রফেসর চেস্টার রেবিয়ার

(Prof. Chester Rebeiro)

কম্পিউটার সায়েন্স এন্ড ইঞ্জিনিয়ারিং

(Department of Computer Science and Engineering)

ইন্ডিয়ান ইনস্টিটিউট অফ টেকনোলজি (Indian Institute of Technology), মাদ্রাজ

সপ্তাহ – 01 বক্তৃতা- 09

xv6 মেমরি ম্যানেজমেন্ট

নমস্কার। আমরা প্রসেসর এবং ভার্সুয়াল মেমরি এবং সেগমেন্টেশন সম্পর্কে মেমরি ম্যানেজমেন্ট স্কিমগুলির পূর্ববর্তী ভিডিওগুলিতে দেখেছি এবং আমরা দেখেছি যে কিভাবে x86 সিস্টেমগুলিতে মেমরি পরিচালিত হয়।

(স্লাইডটাইম পড়ুন: 00:30)

এই ভিডিওতে আমরা xv6 এ মেমরি ম্যানেজমেন্ট দেখব। সুতরাং, xv6 একটি অপারেটিং সিস্টেম যা x86 প্লাটফর্মের জন্য লক্ষ্য করা হয়, তাই x86 প্রসেসরের মেমরি ম্যানেজমেন্টের সম্পর্কিত ভিডিওটি গুরুত্বপূর্ণ হবে। সুতরাং, আমাদের এই বিশেষ ভিডিওতে অনেক xv6 এর উৎস বা সোর্স কোড উল্লেখ করা রয়েছে, তাই রেফারেন্সের জন্য আপনি এটিকে আসলে xv6 সোর্স কোড বুকলেট হিসাবে ব্যবহার (সংস্করণ ৪) করতে পারেন যা এই বিশেষ ওয়েবসাইট থেকে ডাউনলোড করা যায়।

(স্লাইডটাইপ পড়ুন: 01:05)

এখন, শুধু স্মরণ করার জন্য একটি x86 সিস্টেমের দুটি স্তরের মেমরি অনুবাদ রয়েছে। তাই সিপিইউ একটি লজিক্যাল অ্যাড্রেস দেয় যা একটি সেগমেন্টেশন প্লাস ও অফসেট এর অন্তর্ভুক্ত। তারপর, একটি সেগমেন্টেশন ইউনিট রয়েছে যা লজিক্যাল অ্যাড্রেসটিকে একটি লিনিয়ার অ্যাড্রেস এবং একটি পেইজিং ইউনিটে রূপান্তরিত করে যা লিনিয়ার অ্যাড্রেসকে ফিজিক্যাল ঠিকানাতে রূপান্তরিত করে এবং শুধুমাত্র তারপর ফিজিক্যাল মেমোরি বা RAM কে আসলে অ্যাক্সেস করা হয়।

(স্লাইডটাইপ পড়ুন: 01:36)

xv6 এর এই মেমরি ম্যানেজমেন্টের একটি সম্পূর্ণ ভিউ পেতে, আমরা এই নির্দিষ্ট চিত্র দেখেছি। এবং বিশেষ করে, আমরা সেগমেন্টেশন ইউনিট সম্পর্কে দেখেছি যা এই অংশটি এখানে পেইজিং ইউনিট দ্বারা অনুসরণ করা হয়েছে। আমরা বিশেষভাবে সেগমেন্টেশন ইউনিটের মধ্যে দেখেছি যে একটি সেগমেন্ট সিলেক্টর রয়েছে, যা একটি রেজিস্টার যেমন স্ট্যাক সেগমেন্ট, কোড সেগমেন্ট, ডিএস, ইএস, এফএস সেগমেন্ট এবং এই নির্দিষ্ট সেগমেন্টের তালিকা যা GDT বা গ্লোবাল ডেসক্রিপটর টেবিল নামে পরিচিত। আর এই বিশেষ GDT এর মধ্যে সেগমেন্ট ডেসক্রিপটর উপস্থাপন করা হয়। অফসেটের সাথে মিলিত হলে সেগমেন্টের ডেসক্রিপটর আপনাকে লিনিয়ার ঠিকানা দেবে। তাই আসুন xv6 অপারেটিং সিস্টেমে সেগমেন্টেশন কিভাবে পরিচালিত হয় তার সম্পর্কে আরো বিস্তারিত দেখাব। সুতরাং, প্রথমত আমরা সেগমেন্টের ডেসক্রিপটর সম্পর্কে আরও দেখব।

(স্লাইডটাইপ পড়ুন: 02:38)

x86 সিস্টেমের প্রতিটি সেগমেন্ট ডেসক্রিপটর 64 বিট এর হয়। সুতরাং, এই 64 বিটকে প্রতিটি 32 বিটের 2 টি ওয়ার্ডস হিসাবে দেখা যাবে। সেগমেন্ট ডেসক্রিপটরে সেগমেন্টের বেশ কয়েকটি বৈশিষ্ট্য রয়েছে। কিছু গুরুত্বপূর্ণ বৈশিষ্ট্য এখানে দেখানো হয়। সুতরাং, একটি গুরুত্বপূর্ণ বৈশিষ্ট্য হল সেগমেন্ট বেস বা সেগমেন্টের বেস ঠিকানা। সুতরাং, এই বেস ঠিকানা 3 অংশ নিয়ে গঠিত; এটি 32 বিটের এবং 3 অংশ দ্বারা গঠিত, এখানে 16 বিট, 16 থেকে 23 বিট, এবং 24 থেকে 31 বিট বিদ্যমান। সেগমেন্ট লিমিট সেগমেন্টের সীমা ধারণ করে। সুতরাং, এটি 20 বিট এর এবং এর দুটি অংশ রয়েছে। এখানে উপস্থিত রয়েছে 16 টি সর্বনিম্ন উল্লেখযোগ্য বা সিগনিফিক্যান্ট বিট আর রয়েছে 4 টি মোস্ট বা সবচেয়ে উল্লেখযোগ্য বিট। সেগমেন্টের ডেসক্রিপটরের আরেকটি গুরুত্বপূর্ণ অ্যাট্রিবিউট হল বিশেষাধিকার স্তর বা প্রিভিলেজ লেবেল। প্রিভিলেজ লেবেল 2 টি বিটের হয় এবং 0 থেকে 3 এর মান থাকতে পারে। তাই, ব্যবহারকারীর প্রসেসের যাদের সবচেয়ে কম অধিকার রয়েছে তাদের 3 মূল্য দেওয়া হয়, অপরদিকে অপারেটিং সিস্টেম কোড যা সর্বোচ্চ সুবিধাযুক্ত স্তরে রয়েছে তার মান 0। সেগমেন্ট ডেসক্রিপটরের আরেকটি বৈশিষ্ট্যটি হল সেগমেন্টের প্রকার বা সেগমেন্ট টাইপ। সুতরাং, এগুলির 3 মান STA_X থাকতে পারে যা এক্সিকিউটেবল, পঠনযোগ্য সেগমেন্ট এবং সেইসাথে লেখাযোগ্য সেগমেন্ট। এই ছাড়াও, আপনি আসলে একটি নির্দিষ্ট সেগমেন্টের জন্য এই বৈশিষ্ট্যগুলি একত্রিত করতে পারেন। উদাহরণস্বরূপ, আপনার একটি সেগমেন্ট থাকতে পারে, যা এক্সিকিউটেবল এবং পঠনযোগ্য। সুতরাং, আপনি উল্লেখ করতে পারেন যে সেগমেন্টটি একটি টাইপ X এবং সেইসাথে R.

(স্লাইডটাইপ পড়ুন: 04:38)

সুতরাং xv6 তে, এই সেগমেন্টের ডেসক্রিপটর অপারেটিং সিস্টেম কোডের একটি কাঠামো দ্বারা প্রতিনিধিত্ব করে। সুতরাং, যদি আপনি mmu ডট h এ তাকান, তাহলে আপনি আসলে এই বিশেষ কাঠামো দেখতে পাবেন। সুতরাং, আপনি দেখতে পারেন সেগমেন্ট ডেসক্রিপটরের সমস্ত বৈশিষ্ট্যগুলি এই কাঠামো দ্বারা প্রতিনিধিত্ব করা হয়। উদাহরণস্বরূপ, 32 বিটের বেস ঠিকানাটিকে 3 টি অংশে ভাগ করা হয় আর এখানেও 3 টি অংশ দ্বারা দেখানো হয়েছে। আপনার এখানে 16 টি বিট আছে তারপরে আছে 8 টি বিট এবং এখানে সবচেয়ে গুরুত্বপূর্ণ 8 বিট উপস্থিত রয়েছে। এই ছাড়াও, xv6 তে একটি ম্যাক্রো আছে যা SEG নামে পরিচিত, যা 4 টি প্যারামিটার নেয় যেগুলি: টাইপ, বেস বা বেস ঠিকানা, সেগমেন্ট লিমিট এবং ডিপিএল (dpl)। সুতরাং, এই ম্যাক্রোটি একটি সেগমেন্ট ডেসক্রিপটর তৈরি করতে ব্যবহৃত হয়, এটি একটি হেলপার ম্যাক্রো যা সেগমেন্ট ডেসক্রিপটর তৈরি করতে ব্যবহৃত হয়। SEG ম্যাক্রোর একটি সাধারণ ব্যবহার হল এই যে এসইজি এর এই বিশেষ ব্যবহারটি একটি সেগমেন্ট তৈরি করে যার একটি বেস অ্যাড্রেস হল 0, এর সীমা 2 টু দি পাওয়ার 32 বিয়োগ 1 অর্থাৎ $0 \times$ এর পরে 8F পর্যন্ত হয়। এটি একটি ব্যবহারকারীর প্রভিলেজ স্তর আছে, যা dpl আন্ডারস্কোর (underscore) ব্যবহারকারী দ্বারা নির্দিষ্ট এবং এটি টাইপ W- হয় মানে সেগমেন্টটি লিখনযোগ্য হয়।

(স্লাইডটাইপ পড়ুন: 06:00)

xv6 অনেক বেশি সেগমেন্টেশন ব্যবহার করে না, এটি শুধুমাত্র 4 টি বিভাগ তৈরি করে। এগুলি হল কার্নেল কোড, কার্নেল ডেটা, ইউজার কোড এবং ইউজার ডেটা। এই সমস্ত বিভাগের একটি বেস ঠিকানা আছে 0 এবং 4 গিগাবাইটের একটি সীমা আছে। সমস্ত কোড বিভাগগুলি যা কার্নেল কোড এবং ইউজার কোড হয় এক্সিকিউটেবল এবং পঠনযোগ্য অর্থাৎ X, R। ডেটা অর্থাৎ কার্নেল ডেটা এবং ইউজার ডেটা লিখনযোগ্য মানে W হয়। এখন কার্নেল কোড এবং ডেটা এর 0 DPL মান আছে, যা সর্বোচ্চ প্রভিলেজ স্তর নির্দেশ করে যখন ব্যবহারকারীর কোড এবং ডেটার একটি DPL মান থাকে 3 যা নির্দেশ করে এর সর্বনিম্ন প্রভিলেজ স্তর। পরবর্তী আমরা দেখব কিভাবে এই 4 টি সেগমেন্ট xv6 এর মধ্যে তৈরি করা হয়।

(স্লাইডটাইপ পড়ুন: 06:54)

সুতরাং, যখন সেগমেন্টেশন আসে তখন এটি একটি GDT এর দরকার হয়, যা মেমরিতে উপস্থিত। সুতরাং এই বিশেষ ঘোষণা যা স্ট্রাক্ট এসইজিডিইএসসি জিডিটি (struct segdesc gdt) যা সেগমেন্টের সংখ্যা গণনা করে একটি অ্যারে বা GDT- এর অন্তর্গত। সুতরাং যদি আপনি কার্নেল কোডের লাইন সংখ্যা 2308 সন্ধান করেন, তাহলে আপনি এই ঘোষণাটি দেখতে পাবেন। সুতরাং, এই নির্দিষ্ট মেমরি অ্যারে বা এই বিশেষ অ্যারে GDT টেবিল সংরক্ষণ করতে ব্যবহৃত হয়। সুতরাং এই বিশেষ অ্যারে এই কোডগুলি দ্বারা ভরা হয়। তাই, আমরা নিম্নোক্ত 4 টি সেগমেন্ট তৈরি করি। সুতরাং, আমরা 4 টি সেগমেন্ট তৈরি করি যথা কার্নেল কোড সেগমেন্ট, কার্নেল ডেটা সেগমেন্ট, ইউজার কোড সেগমেন্ট এবং ইউজার ডেটা সেগমেন্ট। সুতরাং, এই সেগমেন্টগুলি এখানে

দেখানো নিয়ম অনুযায়ী তৈরি করা হয়। অবশেষে, আমাদের এই জিডিটি রেজিস্টারটি নির্দিষ্ট জিডিটি টেবিলকে নির্দেশ করা দরকার, যাতে এই বিশেষ ফাংশনের আহ্বান সম্পন্ন হয় যাকে বলা হয় lgdt বা লোড (load gdt)। সুতরাং, লোড gdt 512 নম্বর লাইনে উপস্থিত একটি ফাংশন; এবং মূলত এটি দুটি প্যারামিটার গ্রহণ করে, এটি সেগমেন্ট ডেসক্রিপটরের একটি পয়েন্টার নেয় যা মূলত এই জিডিটি টেবিলের পয়েন্টার এবং GDT টেবিলের আকার যা আমরা cgdt হিসাবে পাস করেছি, ওটাই GGT টেবিলের আকার। এবং, মূলত এটি শুধু এই বিশেষ ইন্সট্রাকশন কল lgdt কে আহ্বান করতে যাচ্ছে, যা জিডিটি পয়েন্টার জিডিটি এর ঠিকানা দিয়ে mmu পূরণ করে।

(স্লাইডটাইপ দেখুন: 08:38)

এবার আমরা সেগমেন্টেশন থেকে পেজিং ইউনিটে চলে যাব আর দেখবো কিভাবে xv6 অপারেটিং সিস্টেমে ভার্চুয়াল মেমরি পরিচালিত হয়। যেমন আমরা x86 সিস্টেমে আগে দেখেছি, ভার্চুয়াল ঠিকানা সিস্টেমটি আসলে দুইটি লেবেলের পেজ ডিরেক্টরী দ্বারা পরিচালিত হয়। ফলস্বরূপ, লিনিয়ার ঠিকানা বা ভার্চুয়াল ঠিকানা বিভক্ত হয় 3 অংশে; যথা একটি ডিরেক্টরি এন্ট্রি রয়েছে যা 10 বিটের, 10 বিটের একটি টেবিল এন্ট্রি এবং 12 বিটের একটি অফসেট। ডিরেক্টরি এন্ট্রি ব্যবহার করা হয়, একটি পেজ ডিরেক্টরি টেবিলের মধ্যে সূচী তৈরিতে যা মেমরিতে উপস্থিত থাকে। x86 সিস্টেমে, এই পেজ ডিরেক্টরি CR 3 রেজিস্টার হিসাবে পরিচিত একটি রেজিস্টার দ্বারা নির্দেশিত হয়। এখন, পেজ ডিরেক্টরি এন্ট্রির বিষয়বস্তু তারপর একটি পেজ টেবিলকে নির্দেশ করার জন্য ব্যবহার করা হয়। সুতরাং লিনিয়ার ঠিকানাটির দ্বিতীয় অংশটি অর্থাৎ টেবিল ইন্ডেক্স তারপর পেজ টেবিলের মধ্যে অফসেট করার জন্য ব্যবহার করা হয় তারপর সংশ্লিষ্ট পেজ টেবিলের বিষয়বস্তুর সঙ্গে অফসেট যুক্ত করে সংশ্লিষ্ট ফিজিক্যাল অ্যাড্রেস পাওয়া যায়।

(স্লাইডটাইপ পড়ুন: 09:58)

এখন, আমরা দেখতে পারি কিভাবে এই ভার্চুয়াল অ্যাড্রেসিং স্কিমটি xv6 তে পরিচালিত হয়? যখন xv6 বুট শুরু করে, অপারেটিং সিস্টেম কার্নেল কোড এবং তথ্য র‍্যামের নিচের অঞ্চলে কপি করা হয় যেটা এই গোলাপী রঙের অঞ্চলে প্রদর্শিত। যেহেতু ওএস পেজ ডিরেক্টরি বুট করতে থাকে এবং পেজ টেবিলগুলি তৈরি করা হয় এমনভাবে তাই পুরো RAMটি লজিক্যাল অ্যাড্রেস স্পেসের উচ্চতর অঞ্চলে ম্যাপ হয়ে যায় অর্থাৎ RAM এর 0 অবস্থানটি KERNBASE হিসাবে সংজ্ঞায়িত অর্থাৎ 0x80000000 এর সঙ্গে ম্যাপ করা হয়। একইভাবে, এই RAM-র প্রতিটি ঠিকানাটি লজিক্যাল স্পেসে একটি স্বতন্ত্রভাবে ম্যাপ করা ঠিকানা থাকবে। কার্নেল এবং ফিজিক্যাল র‍্যামের জন্য লজিক্যাল অ্যাড্রেস স্পেসের মধ্যে কেন এমন ওয়ান টু ওয়ান ম্যাপিং তৈরি করা হয়েছে? কারণ যে, এই ধরনের একটি মানচিত্রে কার্নেল খুব সহজেই পরিবর্তিত হয় ভার্চুয়াল অ্যাড্রেস থেকে ভার্চুয়াল থেকে প্রাসঙ্গিক ফিজিক্যাল ঠিকানাতে রূপান্তর করতে পারে যা কোনও ভার্চুয়াল অ্যাড্রেসকে কার্নেল স্পেসে প্রদান করা হয়। অপারেটিং সিস্টেম বা কার্নেল V2P নামে পরিচিত একটি ম্যাক্রো বা ফিজিক্যাল মেমরি কনভার্সন ম্যাক্রোর

ভার্চুয়ালের সাহায্যে প্রাসঙ্গিক ফিজিক্যাল ঠিকানাটি সহজেই পেতে পারে। একইভাবে, ফিজিক্যাল ঠিকানা থেকে সংশ্লিষ্ট ভার্চুয়াল ঠিকানাতে রূপান্তর করা খুব সহজ হবে। তাই, RAM এর যেকোনো ফিজিক্যাল অ্যাড্রেস সংশ্লিষ্ট লজিক্যাল অ্যাড্রেসে রূপান্তরিত করা যায় অথবা ম্যাক্রো সংশ্লিষ্ট ভার্চুয়াল অ্যাড্রেস যা ফিজিক্যাল টু ভার্চুয়াল মেমোরি ম্যাক্রো রূপে পরিচিত সংক্ষেপে P2V বলা হয়। সুতরাং এই দুটি ম্যাক্রোর মধ্যে গুরুত্বপূর্ণ বিষয় হল এই KERNBASE।

(স্লাইডটাইপ পড়ুন: 12:01)

সুতরাং আসুন আরও বিস্তারিতভাবে এই দুটি ম্যাক্রো দেখ যাক। সুতরাং, যদি আপনি এই দুটি ম্যাক্রো দেখেন, তবে এটি কার্নেল সোর্স কোডে লাইন নম্বর 212-এ উপস্থিত থাকতে দেখবেন যে ম্যাক্রো P2V কেবল একটি অ্যাড্রেস 'a' নেয় - যা একটি ফিজিক্যাল ঠিকানা এবং এটি ভার্চুয়াল ঠিকানাতে পরিবর্তন করে একটি KERNBASE যোগ করে। একইভাবে, ম্যাক্রো V2P একটি ভার্চুয়াল অ্যাড্রেস 'a' নেয় এবং KERNBASE কে বিয়োগ করে এটি একটি ফিজিক্যাল ঠিকানাতে রূপান্তরিত করে। সুতরাং, আপনি ভার্চুয়াল থেকে ফিজিক্যাল এবং তদ্বিপরীত অর্থাৎ ফিজিক্যাল থেকে ভার্চুয়াল রূপান্তর অপারেটিং সিস্টেমের কাছে খুবই সহজ তা দেখতে পেলেন।

(স্লাইডটাইপ পড়ুন: 12:47)

এখন আসুন দেখি xv6 অপারেটিং সিস্টেম কীভাবে ফিজিক্যাল ঠিকানা এবং লজিক্যাল অ্যাড্রেস স্পেসের মধ্যে ম্যাপিং তৈরি করে। সুতরাং, এখানে এই বিশেষ চিত্রটি ফিজিক্যাল RAM কে বোঝায়। সুতরাং, আপনি দেখতে পাচ্ছেন যে ফিজিক্যাল RAM বিভিন্ন অঞ্চলে বিভক্ত। 0 থেকে 640 K পর্যন্ত অঞ্চল বেস মেমরি হিসাবে পরিচিত হয়; 640K কে 1 মেগাবাইট অঞ্চল যা 1 এর পরে 5 টি 0 আই/ও বা I / O স্থান হিসাবে পরিচিত। এখন 1 মেগাবাইট থেকে ফিস্টপ পর্যন্ত অঞ্চল হল এক্সটেন্ডেড মেমরি। এখন এই ফিস্টপ একটি ম্যাক্রো হিসাবে সংজ্ঞায়িত করা হয়, যা সিস্টেমের উপস্থিত RAM-র পরিমাণ পরিমাপ করে। সুতরাং, এটি 2 গিগাবাইট, 4 গিগাবাইট বা 16 গিগাবাইট মত কিছু হতে পারে এবং সিস্টেম থেকে সিস্টেম থেকে আলাদা হতে পারে। তাই এই অঞ্চলের উপরের অঞ্চল যেটা 4 GB পর্যন্ত প্রসারিত তা মেমরি ম্যাপ ডিভাইস দ্বারা ব্যবহৃত হয়। এখন, এই নির্দিষ্ট ফিজিক্যাল RAM-র ম্যাপিং তৈরি করার জন্য, অপারেটিং সিস্টেমটি পেজ ডিরেক্টরি এবং পেজ টেবিল তৈরি করবে। সুতরাং, এই ম্যাপ তৈরি করার জন্য, অপারেটিং সিস্টেম কোডটি অনেক ম্যাক্রো সংজ্ঞায়িত করে। সুতরাং, গুরুত্বপূর্ণ কিছু কিছু এখানে দেখানো হয়েছে। সুতরাং, আমরা ইতিমধ্যে KERNBASE সম্পর্কে দেখেছি যা 0x8 এর পর সাতটি 0 এর সমান এবং আরো কয়েকটি ম্যাক্রো আছে যেমন KERNLINK, PHYSTOP এবং EXTMEM। এখন KERNBASE ভার্চুয়াল অ্যাড্রেসটি নির্ধারণ করে, যেখান থেকে কার্নেলের স্পেস শুরু হয়। সুতরাং, যদি আপনি এই লজিক্যাল অ্যাড্রেস স্পেসটি দেখেন, তাহলে আপনি দেখতে পাবেন যে এই বিশেষ অবস্থানে KERNBASE টি সংজ্ঞায়িত হয়েছে। এই অবস্থানের নীচের মেমরির অঞ্চলটি ব্যবহারকারীর স্থান

এবং এই অঞ্চলের ব্যবহারকারী প্রসেসগুলি এক্সিকিউট হয়; আবার এই KERNBASE এর উপরের অঞ্চলের অর্থাৎ এখান থেকে সর্বোচ্চ 4 গিগাবাইট(Gigs) পর্যন্ত হয় যেখানে অপারেটিং সিস্টেম উপস্থিত থাকে পাশাপাশি এটি মেমরি পরিচালনা করে এবং ডিভাইসগুলির সাথে যোগাযোগ করে। এখন, আরেকটি গুরুত্বপূর্ণ ম্যাক্রো হল KERNLINK, যা KERNBASE প্লাস 1 এমবি। সুতরাং, আপনি এই জায়গাটা বিশেষভাবে দেখুন, যেটা হল KERNBASE প্লাস 1 MB অর্থাৎ KERNLINK। সুতরাং, এই অবস্থানটি গুরুত্বপূর্ণ কারণ এটি এমন একটি অবস্থান যেখানে কার্নেল কোড এবং ডেটা উপস্থিত রয়েছে। সুতরাং, KERNLINK এ, একটি কার্নেল টেক্সট অর্থাৎ কার্নেল কোড আছে; এবং কার্নেল কোড সম্পন্ন হওয়ার পরে, যে অবস্থানগুলি কার্নেল ডেটা দ্বারা গঠিত হয় যা গ্লোবাল ডেটা যা xv6 অপারেটিং সিস্টেমে বর্তমান। কার্নেল কোড এবং ডেটা শেষ হয় এই 'এন্ড'(end)চিহ্ন দ্বারা। শেষ থেকে এই বিশেষ অঞ্চল 0xFE আর তারপর ছয়টি 0 পর্যন্ত অবস্থান ফ্রি মেমরি। এই বিশেষ ফ্রী মেমরি অপারেটিং সিস্টেমের বিভিন্ন জিনিস যেমন ওএস হীপ এবং ব্যবহারকারী প্রসেসের পেজের বরাদ্দের জন্য ব্যবহার করা হয়। এখন ফিজিক্যাল RAM- র মধ্যে এই কার্নেলের ম্যাপটি তৈরি করতে একটি গঠন যা কেম্যাপ(kmap) নামে পরিচিত। তাই লাইন সংখ্যা 1823-এ বিদ্যমান kmap কার্টামোটিতে চারটি উপাদান রয়েছে। সুতরাং, এখানে আছে ভিআইআরট(virt), যা ভার্চুয়াল ঠিকানাটির একটি পয়েন্টার। তারপর এটার আছে ফিজিক্যাল স্টার্ট অ্যাড্রেস যেটা ফিজিক্যাল অ্যাড্রেসকে নির্দেশ করে এবং ঐ অঞ্চলের ফিজিক্যাল শেষ আর ভার্চুয়াল অ্যাড্রেসের ম্যাপ হয় এবং অবশ্যই এটা করার অনুমতি আছে। সুতরাং, xv6 এরূপ চারটি অঞ্চলকে সংজ্ঞায়িত করে। সুতরাং, আমাদের I / O স্থান আছে, কার্নেল টেক্সট এবং শুধুমাত্র পঠনযোগ্য তথ্য, কার্নেল ডেটা এবং মেমরি অঞ্চল এবং ডিভাইসের স্থান। সুতরাং, এই চারটি অঞ্চলকে প্রকৃতপক্ষে এই বিশেষ কার্নেল স্পেসে ম্যাপ করা যায়। উদাহরণস্বরূপ, I / O স্পেস যা ভার্চুয়াল অ্যাড্রেস KERNBASE থেকে শুরু করে এবং EXTMEM পর্যন্ত অর্থাৎ 0 পর্যন্ত ফিজিক্যাল ঠিকানাতে ম্যাপ করা হয়। তাই, এটি 1 মেগাবাইট অঞ্চল 0 থেকে শুরু করে I / O স্পেসের শেষ পর্যন্ত যা 1 মেগাবাইট হয়। সুতরাং, এটি I / O স্থান হিসাবে পরিচিত এবং সাধারণত xv6 অপারেটিং সিস্টেম দ্বারা ব্যবহৃত হয় না। KERNLINK এর পরের অঞ্চলটি কার্নেল টেক্সট এবং র ডেটা যা কার্নেল কোড এবং শুধুমাত্র পঠনযোগ্য তথ্য সংরক্ষণ করতে ব্যবহৃত হয়। তাই, এই অঞ্চলটি আর এক্সটেন্ডেড মেমরির শুরু থেকে অর্থাৎ KERNLINK যা এই লাইনের দ্বারা এক্সটেন্ডেড মেমোরির মধ্যে উপস্থিত একটি বিশেষ অঞ্চল এর মধ্যে ম্যাপ করা হয়। সুতরাং, এটি এক্সটেন্ডেড মেমরির শুরুতে রয়েছে যাতে রয়েছে কার্নেল কোড এবং শুধুমাত্র ফিজিক্যাল RAM- এর শুধুমাত্র পঠনযোগ্য ডাটা। তারপর আমাদের ডাটা অঞ্চলের সঙ্গে মুক্ত মেমরি মধ্যে ম্যাপ হয়, এবং পরিশেষে, আপনি ডিভাইস স্পেস পাবেন যা এই বিশেষ অবস্থান 0xFE এর পর ছয়টি 0 এর উপরে হয়।

(স্লাইডটাইপ পড়ুন: 18:38)

আসুন দেখি যে xv6 কার্নেল স্পেসের থেকে র‍্যাম পর্যন্ত ম্যাপিং তৈরির জন্য কতগুলি পদক্ষেপ রয়েছে। সুতরাং, এই চারটি ধাপ জড়িত হয়। প্রথমে, একটি পেজিং সফর্ম করা হয়। সুতরাং, ডিফল্টভাবে যখন সিস্টেমটি চালু করা হয় তখন পেজিংটি অক্ষম থাকে। সুতরাং, পেজিং চালু করার জন্য, একটি নির্দিষ্ট বিট যা পেজিং এনাবল বিট যা CR0 রেজিস্টারে থাকে, সেটিকে 1 সেট করতে হবে। সুতরাং CR0 রেজিস্টার x86 প্রসেসরের একটি নির্দিষ্ট রেজিস্টার এবং সেই রেজিস্টারটিতে আছে একটি পেজিং এনাবল বিট যেটা পেজিংকে সক্রিয় করার জন্য 1 এ সেট করা প্রয়োজন।

(স্লাইডটাইপ পড়ুন: 19:২5)

আরেকটি ধাপ হল পেজ ডিরেক্টরি এবং পেজ টেবিল তৈরি করা। পেজ ডিরেক্টরি তৈরি এবং পূরণ করার জন্য, ফাংশন ওয়াক পেজ ডায়রেক্টরি বা walkpgdir কে অপারেটিং সিস্টেম দ্বারা চালানো হয়। তাই, যদি আপনি সোর্স কোডটি সন্ধান করেন, তাহলে আপনি লাইন নম্বর 1754 এ এই walkpgdir ফাংশনটি দেখতে পাবেন।

(স্লাইডটাইপ পড়ুন: 19:45)

ওয়াক পেজ ডিরেক্টরি ফাংশনটি ভার্সুয়াল ঠিকানাটির সাথে সংশ্লিষ্ট পেজ টেবিলের এন্ট্রি তৈরি করে। সুতরাং, মূলত এটি এই নির্দিষ্ট পেজ ডিরেক্টরির একটি এন্ট্রি তৈরি করতে যাচ্ছে। দ্বিতীয়ত, যদি পেজ ডিরেক্টরির মধ্যে সংশ্লিষ্ট পেজ টেবিল এন্ট্রি উপস্থিত না হয় তবে এটি RAM-র মধ্যে একটি পেজ টেবিল তৈরি করে। তাই, এই পেজ টেবিলটি আমরা দেখেছি যে 1 পৃষ্ঠার হয় আর এটা 4 কিলো বাইটের হতে হবে। সুতরাং, আমরা দেখেছি যে এখানে 1024 টি এন্ট্রি আছে এবং প্রতিটি এন্ট্রি 32 বিটের। সুতরাং, এই সমস্ত পেজ টেবিলের মধ্যে 4 কেবি হবে যা একটি পৃষ্ঠা রাখবে। অনুরূপভাবে, অন্য পেজ টেবিলগুলো তৈরি হয় প্রয়োজন হলে। সুতরাং, এই বিশেষ ফাংশনটি একটি ম্যাক্রো ব্যবহার করে যেমন PDX ম্যাক্রো যা ভার্সুয়াল অ্যাড্রেসটি পেজ ডিরেক্টরীর সূচকে খুঁজে বের করে দেবে, এটি আপনাকে লিনিয়ার ঠিকানাটির উপরের 10 টি বিট দেবে। তারপর আমরা পেজ টেবিল এন্ট্রি ঠিকানা যা পেজ ডিরেক্টরি এন্ট্রি দেবে এবং PTX ম্যাক্রো যা ভার্সুয়াল ঠিকানা নেবে এবং আপনাকে পেজ টেবিল এন্ট্রি দেবে।

(স্লাইডটাইপ পড়ুন: 21:06)

পেজ ডিরেক্টরি তৈরি করার পর, পরবর্তী ধাপে পেজ টেবিলটি পূরণ করা হয়। সুতরাং, এটি ম্যাপ পেজ দ্বারা করা হয় যা xv6 উৎস কোডের লাইন সংখ্যা 1779 তে বিদ্যমান।

(স্লাইডটাইপ পড়ুন: 21:17)

সুতরাং, ম্যাপ পেজগুলি কি এই ভার্চুয়াল ঠিকানা থেকে ফিজিক্যাল ঠিকানার ম্যাপিং সহ এই নির্দিষ্ট পেজ টেবিল পূরণ করতে যাচ্ছে। সুতরাং, এই নির্দিষ্ট টেবিলের এন্ড্রিতে রয়েছে যেমন ফিজিক্যাল ঠিকানা ম্যাপিং, অনুমতি এবং বর্তমান বিট। তাই, আমরা দেখেছি যে পেজ টেবিলের একটি এন্ড্রি সংশ্লিষ্ট ফিজিক্যাল অ্যাড্রেস তৈরি করতে অফসেট সহ ব্যবহার করা হয়।

(স্লাইডটাইপ পড়ুন: 21:47)

সুতরাং, একবার আমরা পেজ ডিরেক্টরি এবং পেজ টেবিল তৈরি করেছি, চূড়ান্ত পদক্ষেপ হল CR3 রেজিস্টারে লোড করা। CR3 রেজিস্টার x86 প্রসেসরের অন্য একটি রেজিস্টার যা পেজ ডিরেক্টরীর পয়েন্টার অন্তর্ভুক্ত করে।

(স্লাইডটাইপ পড়ুন: 22:05)

তাই অন্য অর্থে এই CR3 রেজিস্টার এখানে আছে, যা MMU এর প্রসেসরে উপস্থিত রয়েছে এবং এটি পেজ ডিরেক্টরিতে মেমরি অবস্থান নির্দেশ করে। তাই এখন আমরা দেখেছি কিভাবে xv6 কোড পেজিংকে সক্ষম করে পেজ ডিরেক্টরিগুলি এবং পেজ টেবিল তৈরি করে। আসুন দেখি xv6 অপারেটিং সিস্টেম কিভাবে বিভিন্ন উদ্দেশ্যে মেমরি বরাদ্দ করে। সুতরাং, এই উদ্দেশ্যে ব্যবহারকারী প্রসেসরের জন্য অথবা অপারেটিং সিস্টেমের নিজস্ব পেজগুলি বরাদ্দ করা হতে পারে।

(স্লাইডটাইম পড়ুন: 22:45)

আমরা দেখেছি যে xv6 কোড এবং কেবলমাত্র পঠনযোগ্য তথ্যটি র‍্যামের 0 তম অবস্থানে লোড হয়ে উপরের দিকে উঠেছে। কোড শেষে কেবলমাত্র পঠনযোগ্য ডাটাগুলি এবং PHYSTOP যেটি হলো RAM-র প্রকৃত শেষ মেমরি পর্যন্ত হল মুক্ত মেমোরি। এই ফ্রি মেমরিটি বিভিন্ন উদ্দেশ্যে যেমন ইউজার প্রসেসরের পেজগুলি বা অভ্যন্তরীণ অপারেটিং সিস্টেম বই রাখার প্রয়োজনীয়তাগুলি বরাদ্দ করার জন্য ওএস দ্বারা ব্যবহার করা হয়। সুতরাং, এখন আমরা দেখতে যাব কিভাবে এই মুক্ত মেমরি অপারেটিং সিস্টেম দ্বারা পরিচালিত হয়। তাই মূলত এই ফ্রি মেমরি একটি বড় জাঙ্ক হতে পারে এবং এটি 4 কিলো বাইটের টুকরগুলো বা গ্রনুলারিটি (granularity) এর পৃষ্ঠাগুলি মধ্যে বিভক্ত করা হয়। সুতরাং আমাদের এই বিশেষ ফ্রী মেমরি অঞ্চলে উপস্থিত বিভিন্ন পেজ আছে। এখন, যখন প্রয়োজন একটি পেজ একটি নির্দিষ্ট কলিং ফাংশনে বরাদ্দ করা হবে এবং প্রয়োজনীয়তা অনুসারে ব্যবহৃত হয়। সুতরাং, ব্যবহারের পরে নির্দিষ্ট পেজটি মুক্ত হয়। এখন, পরের জিনিসটি সম্পর্কে আমরা চিন্তা করতে পারি কিভাবে অপারেটিং সিস্টেম বা যেভাবে xv6 OS নির্ধারিত করে যে কোন পেজটি বরাদ্দ করা হবে এবং কিভাবে পেজগুলি মুক্ত করা উচিত? এটি করার জন্য xv6 কোড মুক্ত পেজগুলির একটি লিঙ্ক তালিকা বজায় রাখে। সুতরাং, আপনি এখানে দেখতে পারেন, এই বিশেষ চিত্রটিতে এমন পেজ আছে যা নীল বা হলুদ হয়। তাই, এই সমগ্র

অঞ্চলটি র‍্যামের ফ্রি মেমরি অঞ্চলের সাথে মিলিত, নীল অঞ্চলের একটি পেজ যা ব্যবহারকারীর প্রসেসে ব্যবহৃত হয় বা অপারেটিং সিস্টেম নিজেই ব্যবহার করে, যখন হলুদ পেজগুলি একেবারে মুক্ত অঞ্চল হয়। এখন, একটি ফ্রি লিস্ট হিসেবে পরিচিত একটি পয়েন্টার দ্বারা নির্দেশিত লিঙ্ক ব্যবহার করে সব মুক্ত পেজগুলিকে একসাথে লিঙ্ক করা হয়। তাই ফ্রিলিস্ট, লিঙ্ক লিস্ট এর হেড কে পয়েন্ট করে এবং সমস্ত ফ্রি পেজ এই তালিকার সাথে যুক্ত হয়। এখন একটি পেজ বরাদ্দ করার জন্য ক্যালোক (calloc) ফাংশন ব্যবহার করা হয়। Calloc ফাংশন মূলত তালিকা থেকে একটি মুক্ত পেজ অপসারণ করবে এবং ওই পেজটির ব্যবহার করার জন্য বরাদ্দ করা হবে। একটি পেজ মুক্ত করার জন্য kfree ফাংশন ব্যবহার করা হয়; মূলত kfree ফাংশন মুক্ত পেজটিকে লিস্টের মধ্যে ফিরে যোগ করবে।

(স্লাইডটাইপ দেখুন: ২৫:১৫)

তাই লিঙ্ক লিস্টটি এখানে দেখানো হয়েছে যেখানে আপনার ফ্রিলিস্ট পয়েন্টার আছে যা তালিকাটির শীর্ষে নির্দেশ করে এবং তারপর পরপর মুক্ত পেজগুলির পয়েন্টার থাকে। এখন, স্ট্যান্ডার্ড লিংক তালিকার থেকে আলাদা একটি জিনিস হল যে পরবর্তী পৃষ্ঠাতে পয়েন্টার সংরক্ষণের জন্য কোনও একক মেমরি নেই। উল্লেখ্য এই পেজের প্রত্যেকটি ৪ কিলো বাইটের এবং এই ৪ কিলো বাইট মুক্ত পেজগুলি অন্য যেকোনো ডেটা ব্যবহারের জন্য ব্যবহৃত হয় না এইভাবে এই পেজের ০ তম অবস্থানের মধ্যে পরবর্তী পেজের পয়েন্টার থাকে। একইভাবে, এই পেজের ০ তম স্থানে পরবর্তী পেজের পয়েন্টার থাকবে। সুতরাং আমরা এই তালিকা তৈরি করেছি। তাই ফ্রিলিস্ট প্রথম বা প্রধান নোডের তালিকা এবং তারপর সেই নোডের ০ তম অবস্থানে পরবর্তী পেজকে পয়েন্ট করে এবং যতক্ষণ পর্যন্ত আমরা লিস্টের শেষে পৌঁছাই ততক্ষণ এটা চলে। সুতরাং, এটা দিয়ে আমরা xv6 কিভাবে মেমরি পরিচালনা করে তার শেষে পৌঁছলাম। এটি সর্বোত্তম পরিচালন প্রকল্প নয়, কিন্তু মেমরির পরিচালনা করার জন্য বেশ কয়েকটি অপারেটিং সিস্টেম কী করে তার এটি সম্ভাব্য উপায়। ধন্যবাদ।